

HW6

zhijing huang

2026-10-03

Question 1

I prefer KNN with $k=35$. Its test AUC of 0.9727 is close to those of logistic regression (0.9746) and KNN with $k=45$ (0.9741), indicating comparable discrimination on this test set. Using 35 neighbors averages over enough observations to reduce sensitivity to individual cases while keeping predictions more local than using 45 neighbors. Although it does not achieve the highest AUC, I consider it a reasonable choice given its competitive performance.

```
library(FNN)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
data <- read.csv("data/breast-cancer.csv")
set.seed(43206)
train_id <- sort(sample(seq_len(nrow(data)), floor(0.7 * nrow(data))))
training <- data[train_id, ]
testing <- data[-train_id, ]
predictors <- c("mean_radius", "mean_texture", "mean_smoothness")
X_bar <- colMeans(training[, predictors])
s <- sapply(training[, predictors], sd)
X_train <- scale(training[, predictors], center = X_bar, scale = s)
X_test <- scale(testing[, predictors], center = X_bar, scale = s)
y_train <- training$y
y_test <- testing$y

fit <- glm(y ~ ., data = data.frame(y = y_train, X_train), family = binomial())
p_logistic <- predict(
  fit,
  newdata = data.frame(X_test),
  type = "response"
)
p_knn5 <- knn.reg(
```

```

    train = X_train, test = X_test, y = y_train,
    k = 5, algorithm = "brute"
)$pred

p_knn15 <- knn.reg(
  train = X_train, test = X_test, y = y_train,
  k = 15, algorithm = "brute"
)$pred

p_knn25 <- knn.reg(
  train = X_train, test = X_test, y = y_train,
  k = 25, algorithm = "brute"
)$pred

p_knn35 <- knn.reg(
  train = X_train, test = X_test, y = y_train,
  k = 35, algorithm = "brute"
)$pred

p_knn45 <- knn.reg(
  train = X_train, test = X_test, y = y_train,
  k = 45, algorithm = "brute"
)$pred
roc_logistic <- roc(y_test, p_logistic, levels = c(0, 1),
  direction = "<", quiet = TRUE)

roc_knn5 <- roc(y_test, p_knn5, levels = c(0, 1),
  direction = "<", quiet = TRUE)

roc_knn15 <- roc(y_test, p_knn15, levels = c(0, 1),
  direction = "<", quiet = TRUE)

roc_knn25 <- roc(y_test, p_knn25, levels = c(0, 1),
  direction = "<", quiet = TRUE)

roc_knn35 <- roc(y_test, p_knn35, levels = c(0, 1),
  direction = "<", quiet = TRUE)

roc_knn45 <- roc(y_test, p_knn45, levels = c(0, 1),
  direction = "<", quiet = TRUE)
roc_test <- list(roc_logistic, roc_knn5, roc_knn15,
  roc_knn25, roc_knn35, roc_knn45)
model_names <- c(
  "Logistic regression",
  "KNN, k = 5",
  "KNN, k = 15",
  "KNN, k = 25",
  "KNN, k = 35",
  "KNN, k = 45"
)

auc_test <- c(
  as.numeric(auc(roc_logistic)),

```

```

as.numeric(auc(roc_knn5)),
as.numeric(auc(roc_knn15)),
as.numeric(auc(roc_knn25)),
as.numeric(auc(roc_knn35)),
as.numeric(auc(roc_knn45))
)

data.frame(
  Model = model_names,
  Test_AUC = round(auc_test, 4)
)

```

```

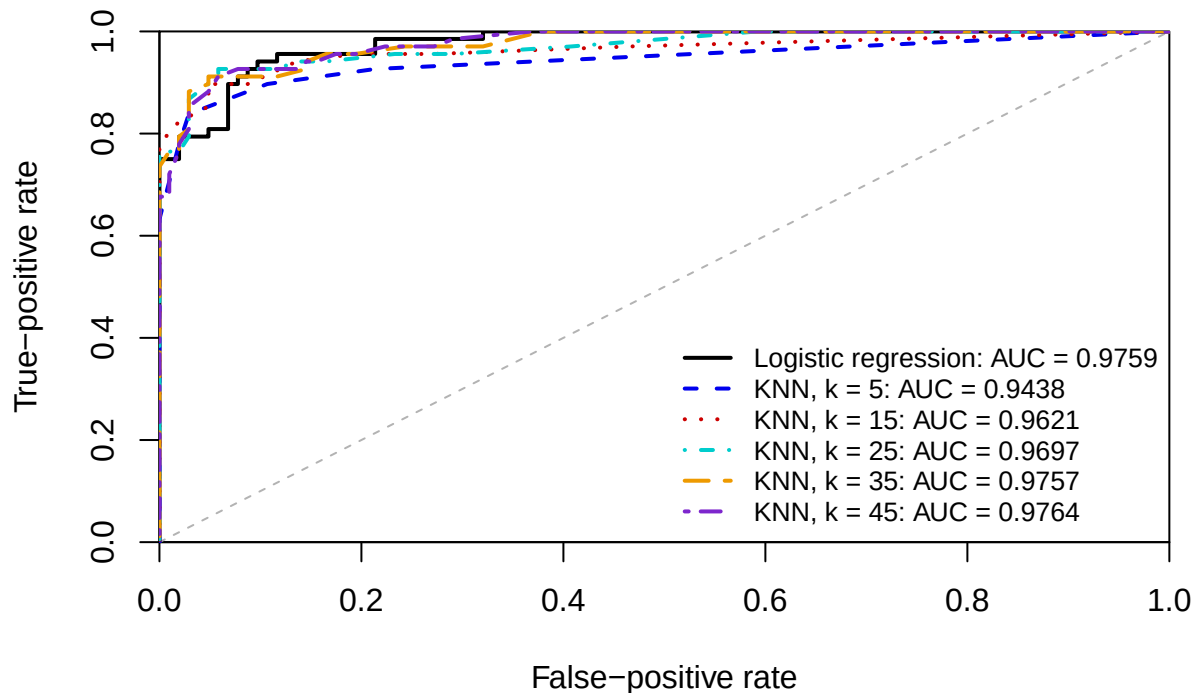
##           Model Test_AUC
## 1 Logistic regression 0.9759
## 2           KNN, k = 5 0.9438
## 3           KNN, k = 15 0.9621
## 4           KNN, k = 25 0.9697
## 5           KNN, k = 35 0.9757
## 6           KNN, k = 45 0.9764

```

```

#plot
colors <- c("black", "blue2", "red3", "cyan3", "orange2", "purple3")
line_types <- c(1, 2, 3, 4, 5, 6)
plot(c(0, 1), c(0, 1), type = "n", xaxs = "i", yaxs = "i",
     xlab = "False-positive rate", ylab = "True-positive rate")
abline(0, 1, col = "gray70", lty = 2)
for (m in seq_along(model_names)) {
  lines(1 - roc_test[[m]]$specificities, roc_test[[m]]$sensitivities,
       col = colors[m], lty = line_types[m], lwd = 2)
}
legend("bottomright", sprintf("%s: AUC = %.4f", model_names, auc_test),
     col = colors, lty = line_types, lwd = 2, cex = 0.8, bty = "n")

```



Question 2

The difference between training and test performance is, as k increases, the training gap shrinks. The training AUC doesn't decrease at every step. Although larger neighbours smoothed the fitted probabilities, it will still act by taking the nearest neighbour, meaning the ranking of both neighbours could take part in one step. At $k = 1$, the overall training performance can be overestimated. New observations cannot use their own labels, and their nearest training neighbors may have different outcomes. Therefore, test performance provides a more realistic assessment of how well the model generalizes.

```
k_values <- c(1, 3, 5, 10, 20)

results <- lapply(k_values, function(k) {
  train_pred <- knn.reg(train = X_train, test = X_train,
                        y = y_train, k = k, algorithm = "brute")$pred
  test_pred <- knn.reg(train = X_train, test = X_test,
                       y = y_train, k = k, algorithm = "brute")$pred

  train_roc <- roc(y_train, train_pred,
                  levels = c(0, 1), direction = "<", quiet = TRUE)
  test_roc <- roc(y_test, test_pred,
                 levels = c(0, 1), direction = "<", quiet = TRUE)

  list(train_pred = train_pred, test_pred = test_pred,
        train_roc = train_roc, test_roc = test_roc,
        train_auc = as.numeric(auc(train_roc)), test_auc = as.numeric(auc(test_roc)))
})
```

```

})

p_train <- do.call(cbind, lapply(results, `[`, "train_pred"))
p_test  <- do.call(cbind, lapply(results, `[`, "test_pred"))
roc_train <- lapply(results, `[`, "train_roc")
roc_test  <- lapply(results, `[`, "test_roc")

auc_train <- sapply(results, `[`, "train_auc")
auc_test  <- sapply(results, `[`, "test_auc")

data.frame(
  k = k_values,
  Training_AUC = round(auc_train, 4),
  Test_AUC = round(auc_test, 4)
)

```

```

##      k Training_AUC Test_AUC
## 1  1      1.0000  0.8900
## 2  3      0.9893  0.9427
## 3  5      0.9863  0.9438
## 4 10      0.9844  0.9557
## 5 20      0.9826  0.9602

```

```

old_par <- par(mfrow = c(2, 3), mar = c(4, 4, 3, 1))

for (m in seq_along(k_values)) {
  plot(
    1 - roc_train[[m]]$specificities,
    roc_train[[m]]$sensitivities,
    type = "l",
    col = "blue",
    lwd = 2,
    xlim = c(0, 1),
    ylim = c(0, 1),
    xlab = "False-positive rate",
    ylab = "True-positive rate",
    main = paste("k =", k_values[m])
  )
  lines(
    1 - roc_test[[m]]$specificities,
    roc_test[[m]]$sensitivities,
    col = "red",
    lty = 2,
    lwd = 2
  )
  abline(a = 0, b = 1, col = "gray", lty = 3)

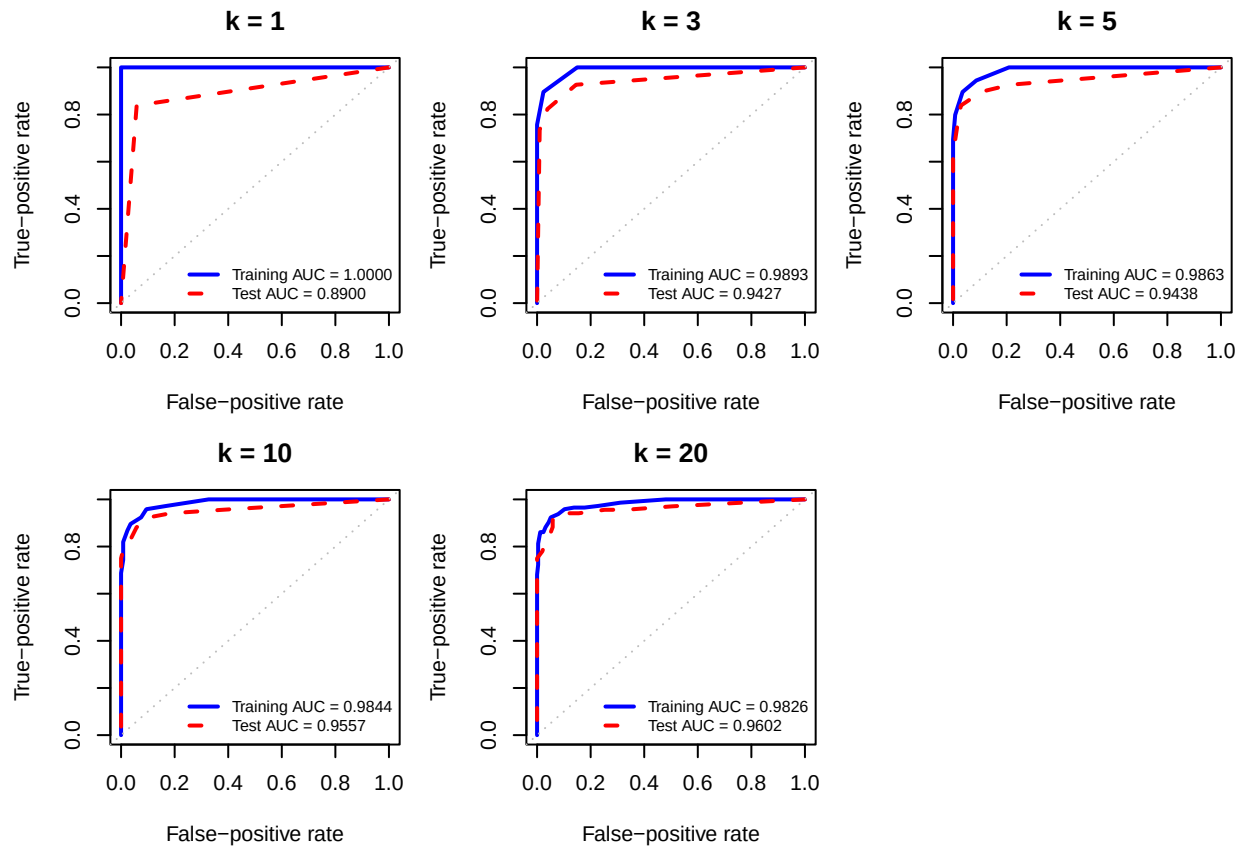
  legend(
    "bottomright",
    legend = c(
      sprintf("Training AUC = %.4f", auc_train[m]),
      sprintf("Test AUC = %.4f", auc_test[m])
    ),
  ),

```

```

col = c("blue", "red"),
lty = c(1, 2),
lwd = 2,
cex = 0.75,
bty = "n"
)
}
par(old_par)

```



Question 3

<https://github.com/r Ruiz35/stat432-fall2026/issues/3>